

Laboratorio 6

Fabián Morales

Transfer learning en PyTorch

`torchvision.models.vgg16` construye la arquitectura VGG-16 y su parámetro `weights` decide la inicialización: `None` la deja aleatoria y `VGG16_Weights.IMAGENET1K_V1` carga los pesos entrenados en ImageNet, con 71.6 % de accuracy top-1. `VGG16_Weights` es un enum que guarda esos pesos junto con sus metadatos, y `weights.transforms` devuelve el preprocesamiento con el que fueron entrenados: `Resize` a 256, `CenterCrop` a 224, escala a `[0, 1]` y normalización con la media 0.485, 0.456, 0.406 y la desviación 0.229, 0.224, 0.225 de ImageNet.

El modelo tiene tres partes. `model.features` agrupa 13 convoluciones 3x3 con ReLU y 5 MaxPool que forman 5 bloques de 64, 128, 256, 512 y 512 canales. `model.avgpool` es un `nn.AdaptiveAvgPool2d` que siempre entrega un mapa de 512x7x7, lo que permite cambiar la resolución de entrada sin romper el clasificador. `model.classifier` tiene tres capas lineales de 25,088 a 4,096, de 4,096 a 4,096 y de 4,096 a 1,000 con ReLU y Dropout de 0.5; para CIFAR-10 se reemplaza la última por una de 10 salidas.

Congelar una capa es asignar `requires_grad = False` a sus parámetros: `autograd` deja de calcular su gradiente, el optimizador no la modifica y el backward se detiene en la primera capa entrenable, lo que ahorra memoria y cómputo. Un bloque completo se congela o descongela con `requires_grad_` sobre el slice de `model.features`. `model.train` activa Dropout y las estadísticas por lote de BatchNorm, `model.eval` los fija para inferencia, y `torch.no_grad` evita construir el grafo durante la evaluación. Para usar learning rates distintos por capa, el optimizador recibe una lista de `param_groups`, cada uno con sus parámetros y su propio lr.

Para medir el costo se usaron `torchinfo.summary`, que muestra por capa la salida, los parámetros y si son entrenables, y `thop.profile`, que devuelve los MACs de un forward. Un MAC es una multiplicación más una suma, así que un MAC equivale a 2 FLOPs, y el backward cuesta cerca del doble del forward. Los parámetros totales definen el tamaño y los FLOPs de inferencia; los entrenables definen cuántos gradientes y estados del optimizador se guardan. En GPU, `torch.cuda.max_memory_allocated` devuelve el pico de memoria de tensores desde el último `reset_peak_memory_stats`, y `torch.cuda.synchronize` espera a que terminen los kernels, lo cual es indispensable para cronometrar porque CUDA es asíncrono.

Datos y preparación

CIFAR-10 tiene 60,000 imágenes RGB de 32x32 en 10 clases balanceadas, con 5,000 por clase en entrenamiento y 1,000 en test. Su media por canal, 0.491, 0.482 y 0.447, es cercana a la de ImageNet, pero su desviación, 0.247, 0.243 y 0.262, es mayor. Se esperaba que cat y dog,

automobile y truck, deer y horse, y airplane con ship fueran los pares más difíciles. El entrenamiento se dividió de forma estratificada en 45,000 imágenes de entrenamiento y 5,000 de validación, y el test se usó una sola vez al final. La CNN recibe las imágenes a 32x32 normalizadas con las estadísticas de CIFAR-10, y VGG-16 las recibe a 224x224, su resolución nativa, con la normalización de ImageNet, lo que multiplica por 49 los píxeles y el cómputo por imagen. Solo en entrenamiento se aplicó RandomCrop con padding de 4 y volteo horizontal, que cambian posición y orientación sin cambiar la clase; validación y test no se alteran para que sus métricas sean estables y comparables. Se usó una GPU NVIDIA GeForce RTX 5070 Ti de 16 GB, un CPU AMD Ryzen 7 7800X3D y precisión mixta bfloat16.

Iteraciones de entrenamiento

Se entrenaron 14 iteraciones. La CNN apila bloques de convolución 3x3, BatchNorm y ReLU. El feature extractor congela los 5 bloques convolucionales y entrena las tres capas densas, con la última reemplazada por una de 10 salidas. El fine-tuning usa SGD con Nesterov, cosine annealing, augmentation, learning rate de 0.01 en el clasificador y uno menor en el backbone mediante param_groups; todas las iteraciones de VGG-16 duran 8 epochs. F4, a 128x128, y T5, con el learning rate del backbone igual al del clasificador, son exploratorias y no compiten; la mejor iteración de cada modelo se eligió por F1 macro de validación. Los tiempos son del paso de entrenamiento.

Tabla 1. Resultados de las iteraciones

Iter.	Configuración	Bloques congelados	Params entren. M	Mejor epoch	Val acc	Val F1	s por epoch	Total s	Pico MB
C1	3 bloques 32 a 128, sin regularizar, Adam, 20 ep	ninguno	0.62	12/20	0.7620	0.7619	0.8	16	805
C2	3 bloques 64 a 256, BN, dropout, aumento, 30 ep	ninguno	2.20	30/30	0.9034	0.9033	1.3	40	950
C3	arquitectura de C2, SGD OneCycle, 40 ep	ninguno	2.20	40/40	0.9264	0.9262	1.2	49	942
C4	4 bloques 64 a 512, SGD OneCycle, 40 ep	ninguno	5.21	40/40	0.9314	0.9315	1.6	65	993
F1	Adam 1e-3 fijo, sin aumento	1 a 5	119.59	8/8	0.8826	0.8835	32.5	260	6,354
F2	Adam 1e-4 cosine, aumento	1 a 5	119.59	8/8	0.9084	0.9084	31.0	248	6,354
F3	SGD 1e-2 cosine, wd 5e-4, aumento	1 a 5	119.59	8/8	0.9038	0.9037	29.2	233	5,898
F4	F2 con entrada 128x128, exploratoria	1 a 5	119.59	8/8	0.8992	0.8991	13.1	105	3,851
T1	descongela b5, lr backbone 1e-3	1 a 4	126.67	8/8	0.9266	0.9264	32.1	257	5,952
T2	descongela b4 y b5, lr backbone 1e-3	1 a 3	132.57	6/8	0.9376	0.9377	41.6	333	5,997
T3	descongela b3 a b5, lr backbone 1e-3	1 a 2	134.04	8/8	0.9460	0.9460	51.3	411	6,011
T4	descongela b4 y b5, lr backbone 1e-4	1 a 3	132.57	6/8	0.9204	0.9201	41.1	329	5,997
T5	descongela b4 y b5, lr backbone 1e-2, control	1 a 3	132.57	8/8	0.9490	0.9490	41.2	330	5,997
T6	descongela b3 a b5, lr backbone 3e-3	1 a 2	134.04	8/8	0.9500	0.9500	55.4	443	6,011

C1 sobreajusta: su pérdida de validación sube de 0.71 a 1.63 mientras la de entrenamiento baja a 0.05, y BatchNorm, dropout y augmentation cierran esa brecha. En el feature extractor, F1 sobreajusta desde la epoch 3 porque sus capas densas de 120 millones de parámetros memorizan con facilidad, y F2 lo corrige con un learning rate menor y augmentation.

Comparación de rendimiento y recursos

El cómputo de entrenamiento se estimó como FLOPs por imagen por imágenes por epochs, donde cada imagen cuesta 2 veces los MACs totales en el forward más 4 veces los MACs de las capas entrenables en el backward, porque el gradiente no baja de la primera capa descongelada. La latencia se midió con lote de 1 en bfloat16 y el throughput con lote de 256.

Tabla 2. Comparación de rendimiento y recursos

Métrica	CNN desde cero	VGG-16 feature extractor	VGG-16 fine-tuning
Iteración y resolución	C4, 32x32	F2, 224x224	T6, 224x224
Parámetros totales M	5.21	134.30	134.30
Parámetros entrenables M	5.21	119.59	134.04
GMACs y GFLOPs por imagen	0.21 y 0.42	15.47 y 30.93	15.47 y 30.93
Latencia GPU ms por imagen	0.79	1.28	1.28
Throughput img/s, lote 256	96,649	1,465	1,483
Tiempo por epoch s	1.6	31.0	55.4
Epoch de mejor val / total	40 / 40	8 / 8	8 / 8
Tiempo total s	65	248	443
Memoria pico GPU MB	993	6,354	6,011
Cómputo total de entrenamiento PFLOPs	2.3	11.3	26.6
Test accuracy	0.9292	0.8953	0.9420
Test precision	0.9291	0.8953	0.9418
Test recall	0.9292	0.8953	0.9420
Test F1	0.9291	0.8952	0.9419
Test F1 con 10 %	0.7253	0.8407	0.8850
Caída de F1 en puntos	20.4	5.5	5.7

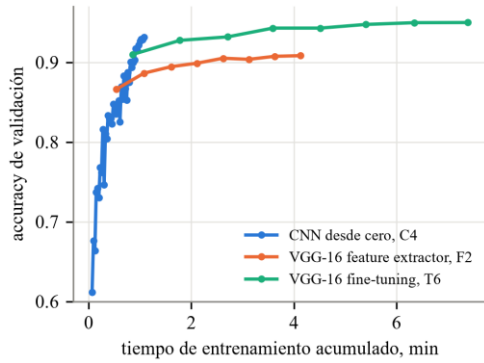


Figura 1. Accuracy de validación contra tiempo

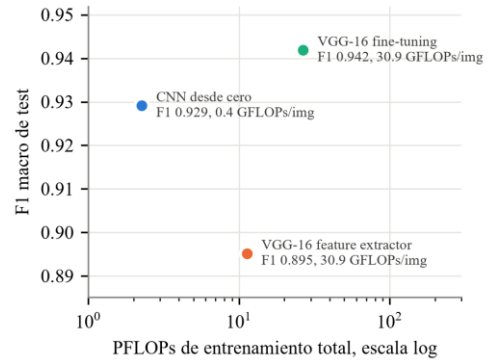


Figura 2. F1 de test contra FLOPs

Análisis de resultados

VGG-16 con fine-tuning obtuvo el mejor desempeño en test, 94.2 % de accuracy y 0.942 de F1, frente a 92.9 % de la CNN y 89.5 % del feature extractor. Su ventaja sobre la CNN es de 1.3 puntos, bajar los errores de 7.1 % a 5.8 %, a cambio de 26 veces más parámetros, 74 veces más FLOPs por imagen, 12 veces más cómputo de entrenamiento y casi 7 veces más tiempo, así que solo se justifica cuando la máxima accuracy importa más que el costo. La CNN desde cero ofrece la mejor relación: en la figura 2 ocupa la esquina superior izquierda y en la figura 1 supera el 90 % de validación a los 48 segundos, mientras que el feature extractor tarda 4 minutos en llegar a 90.8 % y queda dominado, con más costo y menor F1. Con lote de 1 la latencia solo difiere 1.6 veces porque una imagen no llena la GPU; con lote de 256 el throughput difiere 66 veces.

Las features de ImageNet siguen siendo útiles a 32x32 porque los primeros bloques detectan bordes, colores, texturas y partes, patrones genéricos presentes en CIFAR-10, y al ampliar a 224x224 el objeto queda a una escala parecida a la de ImageNet. Pero la imagen ampliada no tiene detalle nuevo y los bloques superiores, especializados en detalles finos, no encajan bien;

por eso el feature extractor rinde menos que la CNN y el fine-tuning, que adapta esos bloques, rinde más. El redimensionamiento multiplica por 49 los FLOPs y la memoria de activaciones, y F4 muestra que a 128x128 se pierde solo 0.9 puntos con menos de la mitad del tiempo por epoch.

En fine-tuning, descongelar más bloques mejoró de forma consistente: 92.6 % con el bloque 5, 93.8 % con los bloques 4 y 5 y 94.6 % con los bloques 3 a 5, con el tiempo por epoch subiendo de 32 a 51 s y la memoria casi igual. Un learning rate de backbone de 0.0001 dejó el modelo en 92.0 %, por debajo incluso de descongelar solo el bloque 5, y uno de 0.01 subió la accuracy pero produjo la mayor brecha entre pérdida de entrenamiento y de validación, 0.04 frente a 0.17, una señal de overfitting. No hubo catastrophic forgetting: la accuracy de la primera epoch siempre superó a la del feature extractor y la pérdida de validación no tuvo un aumento sostenido. El mejor equilibrio fue 0.003 sobre los bloques 3 a 5.

Con el 10 % de los datos la CNN fue la más afectada: su F1 cayó 20.4 puntos, frente a 5.5 del feature extractor y 5.7 del fine-tuning, que la superan por más de 11 puntos. Coincide con lo visto en clase: con pocos datos y dominio similar conviene feature extraction, con una cantidad media conviene fine-tuning de los bloques superiores, y entrenar desde cero solo compete con suficientes datos o un dominio muy distinto.

Los tres modelos comparten el par más confundido, cat y dog, con 140, 193 y 134 errores, seguido de automobile y truck, pero no en la misma proporción: el feature extractor duplica la confusión entre vehículos y confunde más bird con deer, la CNN destaca en cat con frog y airplane con ship, y el fine-tuning comete 580 errores frente a 708 y 1,047, aunque confunde más dog con horse que la CNN.

En un servidor con GPU se elegiría VGG-16 con fine-tuning, por su mejor accuracy y una latencia de 1.3 ms. En un móvil se elegiría la CNN desde cero, que ocupa unos 21 MB frente a más de 500 MB de VGG-16 y pierde solo 1.3 puntos. Como alternativa preentrenada para el móvil se considerarían MobileNetV3 o EfficientNet-B0, con unos 5 millones de parámetros y menos de 0.4 GMACs.

Conclusiones

1. El fine-tuning de los bloques 3 a 5 de VGG-16 fue el modelo más preciso, con 94.2 % de accuracy en test, pero solo supera por 1.3 puntos a la CNN desde cero a cambio de 74 veces más FLOPs por imagen y 12 veces más cómputo de entrenamiento.
2. Con los 45,000 datos de entrenamiento, la CNN desde cero ofrece la mejor relación entre desempeño y recursos y supera al feature extractor, cuyas features congeladas no se adaptan a imágenes de 32x32 ampliadas.
3. El valor del transfer learning crece cuando faltan datos: con el 10 % del entrenamiento la CNN pierde 20.4 puntos de F1, mientras que los modelos preentrenados pierden menos de 6 y la superan por más de 11.

Repositorio: <https://github.com/FabianMoraes/DeepLab6>